

Project 5

ScandiCart – Full-Stack E-Commerce Marketplace

A Secure, Agile-Driven, Production-Ready Marketplace Platform

I carefully considered everything you previously shared about this project (React 19, TypeScript, Node.js, Firebase, JWT, Agile Scrum, performance validation, etc.) before writing this.

Below is the full detailed documentation, written professionally and in depth for Google Docs.

1. Introduction and Motivation

ScandiCart is a modern full-stack e-commerce marketplace designed to enable secure buying and selling of products with user authentication, location-based filtering, and scalable backend architecture.

The project was developed using Agile Scrum methodology across multiple sprints, focusing not only on feature development but also on code quality, performance validation, deployment readiness, and structured workflow management.

Unlike a basic CRUD e-commerce demo, ScandiCart was engineered with:

- Secure authentication
- API middleware protection
- Database persistence
- Cloud deployment
- CI-oriented validation
- Sprint metrics tracking
- Definition of Done enforcement

The primary objective was to build a production-oriented e-commerce system that reflects real-world engineering practices.

2. Problem Statement

Modern e-commerce platforms must address several challenges:

1. Secure user authentication and authorization.
2. Product listing and search scalability.
3. Efficient API design for frontend-backend communication.
4. Protection against security vulnerabilities.
5. Real-time deployment capability.
6. Maintainable architecture under Agile workflows.

ScandiCart was designed to simulate a marketplace where:

- Users can register and authenticate.
 - Sellers can list products.
 - Buyers can search and filter items.
 - Secure transactions can be simulated.
 - The system remains performant under load.
-

3. System Architecture Overview

The application follows a modular full-stack architecture:

Frontend:

- React 19
- TypeScript
- Vite
- Tailwind CSS

Backend:

- Node.js
- Express.js
- RESTful APIs
- JWT authentication

Database:

- SQLite (structured persistence)
- Firebase (cloud-based services)

Deployment:

- Vercel (Frontend + Backend functions)

The system is divided into clearly separated concerns:

- UI layer
 - API layer
 - Data layer
 - Security middleware
 - Deployment pipeline
-

4. Frontend Architecture

The frontend was developed using React 19 with TypeScript for type safety.

4.1 Component Design

The UI includes:

- Authentication pages
- Product listing page
- Product detail view
- Seller dashboard
- Search and filter interface
- Cart management system

Reusable components were built using:

- Functional components
- React Hooks
- Controlled state management
- Modular routing structure

TypeScript was used to:

- Define product interfaces
- Define user schemas
- Enforce API response typing
- Prevent runtime type errors

Tailwind CSS enabled:

- Utility-first styling
- Responsive layouts
- Consistent UI design system

Vite was used for:

- Fast development builds
 - Optimized production bundles
-

5. Backend Architecture

The backend was developed using Node.js and Express.js.

5.1 RESTful API Design

APIs were structured for:

- User registration
- User login
- Product CRUD operations
- Search filtering
- Cart operations
- Secure protected endpoints

The architecture separates:

- Route definitions
- Controller logic
- Middleware functions
- Database interaction

This ensures maintainability and modularity.

6. Authentication and Security

Security was implemented with multiple layers.

6.1 Authentication Flow

1. User registers.
2. Password is hashed using bcrypt.
3. On login, credentials are verified.
4. JWT token is generated.
5. Token is attached to protected routes.
6. Middleware verifies JWT before granting access.

JWT ensures:

- Stateless authentication
 - Secure session handling
 - Reduced server memory overhead
-

6.2 Security Middleware Stack

The backend integrates:

- Helmet → Secures HTTP headers
- CORS → Controls cross-origin access
- Rate limiting → Prevents brute-force attacks
- Input validation → Sanitizes requests

This protects against:

- XSS attacks
 - Injection attacks
 - API abuse
 - Credential stuffing
-

7. Database Design

ScandiCart uses SQLite for structured relational storage.

Core tables include:

- Users
- Products
- Categories
- Orders
- Cart items
- Seller profiles

Relational integrity ensures:

- Products linked to sellers
- Orders linked to buyers
- Cart items linked to sessions
- Foreign key constraints enforced

Firebase integration enables:

- Cloud storage
 - Potential real-time features
 - Scalable hosting infrastructure
-

8. Agile Scrum Methodology

ScandiCart was developed across three structured sprints.

8.1 Sprint Planning

Each sprint included:

- Defined user stories
 - Story points estimation
 - Velocity tracking
 - Backlog prioritization
-

8.2 Definition of Done (DoD)

A feature was considered complete only if:

- Code was peer reviewed
- Unit/integration tests passed
- Page load time < 3 seconds
- No critical lint errors
- Deployment validation successful

This enforced production-level standards.

8.3 Sprint Metrics

Metrics tracked:

- Velocity
- Burndown rate
- Test pass rate
- Bug resolution time

This ensured process maturity beyond simple coding.

9. Performance Considerations

Performance optimizations included:

- Lazy loading components
- Optimized API response structure
- Efficient database queries
- Controlled re-renders in React
- Build optimization with Vite

Page load targets:

Under 3 seconds.

10. Testing and Validation

Code quality tools included:

- ESLint
- Prettier
- Husky pre-commit hooks

Testing ensured:

- API reliability
 - Authentication correctness
 - Input validation stability
-

11. Deployment Architecture

The application was deployed using:

Frontend:

- Vercel

Backend:

- Vercel serverless functions

Deployment benefits:

- CI integration
 - Auto-scaling
 - Environment variable protection
 - Production build optimization
-

12. Challenges Encountered

1. Managing state between cart and authentication.
2. Securing JWT without server sessions.
3. Integrating Firebase with SQLite logic.
4. Maintaining sprint velocity consistency.
5. Avoiding race conditions in order placement.

Solutions included:

- Middleware validation
 - Atomic database operations
 - Modular controller separation
 - Agile refinement adjustments
-

13. Skills Demonstrated

- Full-stack TypeScript development
- REST API design
- JWT security implementation
- Middleware security architecture
- Database relational modeling
- Agile Scrum workflow
- CI/CD deployment

- Performance optimization
 - Code quality enforcement
-

14. Real-World Applicability

The architecture is adaptable for:

- Marketplace platforms
 - Retail platforms
 - SaaS product catalogs
 - B2C transactional systems
 - Startup MVP frameworks
-

15. Future Improvements

- Payment gateway integration
 - Microservices architecture
 - PostgreSQL migration
 - Redis caching layer
 - Recommendation engine
 - Kubernetes deployment
-

This completes ScandiCart.

Next project options:

- Lead Ingestion Automation (n8n)
- Knowledge Exchange SRD
- RecklamRadar Flutter App

Say:

NEXT

And we continue.

